C语言**不会**自动检测数据溢出，这是它与其他现代语言最显著的区别之一。下面分层次详细说明：

一、C语言：不检测，后果自负

1. 整数溢出

- **有符号整数溢出**：属于**未定义行为（Undefined Behavior, UB）**。编译器可以假设它永远不会发生，并据此做激进优化。实际表现可能是回绕、崩溃、产生错误结果，甚至"时间旅行优化"（看起来在溢出发生前就产生了错误）。
- **无符号整数溢出**：定义为**模运算回绕**（wrap-around），即自动对 2^n 取模。这是明确定义的，但通常不是程序员想要的。

```
int a = INT_MAX;
a++;  // UB! 可能回绕、崩溃、被优化掉，什么都可能发生

unsigned int b = UINT_MAX;
b++;  // 明确定义为回绕到 0
```

2. 数组越界/缓冲区溢出

- **完全不检测。**访问 `arr[100]` 当数组只有 10 个元素时，编译器不会报错，运行时也不会自动检查。这是安全漏洞（如栈溢出攻击）的主要来源。

3. 为什么这样设计?

C语言诞生于1970年代，设计哲学是**"信任程序员"**和**零成本抽象**——不为不需要的功能付出运行时性能代价。检测溢出需要额外的指令和分支，C选择不做。

二、其他主流语言的处理方式

语言	整数溢出	数组/缓冲区越界	备注
C++	同C（不检测）	同C（不检测）	但标准库提供 `std::vector::at()` 会检查边界；有 sanitizers 等工具
Java	**静默回绕**（模运算）	**抛异常**（`ArrayIndexOutOfBounds`）	整数溢出不报错但结果错误，这是常见bug来源
Python	**不会溢出**（自动转大整数）	**抛异常**（`IndexError`）	`int` 是任意精度，只有内存限制
JavaScript	Number 按 IEEE 754 处理；BigInt 不会溢出	越界访问返回 `undefined`，不会抛异常	`2**53` 后整数精度丢失；`BigInt` 可安全处理大整数
Rust	**Debug 模式 panic**；Release 默认回绕，但提供丰富 API	**panic**（`index out of bounds`）	可用 `checked_add`、`saturating_add`、`wrapping_add` 显式控制行为
Go	**静默回绕**（模运算）	**panic**（`index out of range`）	与 Java 类似，整数溢出静默但数组越界报错
C#	默认回绕；`checked` 块内抛 `OverflowException`	**抛异常**（`IndexOutOfRangeException`）	可通过编译器开关全局启用检查
Swift	**默认 panic**（运行时错误）	**默认 panic**	需要显式使用 `&+`、`*` 等"溢出运算符"才能回绕
Ada	**默认抛异常**（`Constraint_Error`）	**默认抛异常**	以安全著称，溢出检测是语言核心特性
Haskell	`Integer` 任意精度不溢出；`Int` 回绕	越界是运行时错误	默认使用 `Integer`，安全但较慢
Ruby	**不会溢出**（自动转 Bignum）	**抛异常**（`IndexError`）	与 Python 类似

三、关键对比总结

```
检测严格度（高 → 低）：

Ada / Swift / Rust(Debug)  →  严格检测，溢出即报错
Python / Ruby / Haskell    →  自动扩展精度，理论上不会溢出
Java / Go / C#(unchecked)  →  整数静默回绕（易出bug但不崩溃），数组越界报错
C / C++                    →  完全不检测，未定义行为或静默破坏内存
```

四、C语言程序员该怎么办?

C语言不保护你，但你可以自我保护：

1. **编译器工具**：GCC/Clang 有 `-ftrapv`（有符号溢出陷阱，但支持有限）、`-fsanitize=undefined`（UBSan，检测UB并报告）、`-fsanitize=address`（ASan，检测内存越界）。
2. **手动检查**：运算前判断边界，如 `if (a > INT_MAX - b) /* 溢出 */`。
3. **使用安全库**：如 `<stdckdint.h>`（C23）提供 `ckd_add()` 等函数，返回是否溢出。
4. **选择更安全语言**：如果安全性优先于极致性能，考虑 Rust、Ada 或现代C++。

一句话总结：C语言把溢出的检测责任完全交给了程序员和工具链，编译器和运行时都不会帮你兜底。